

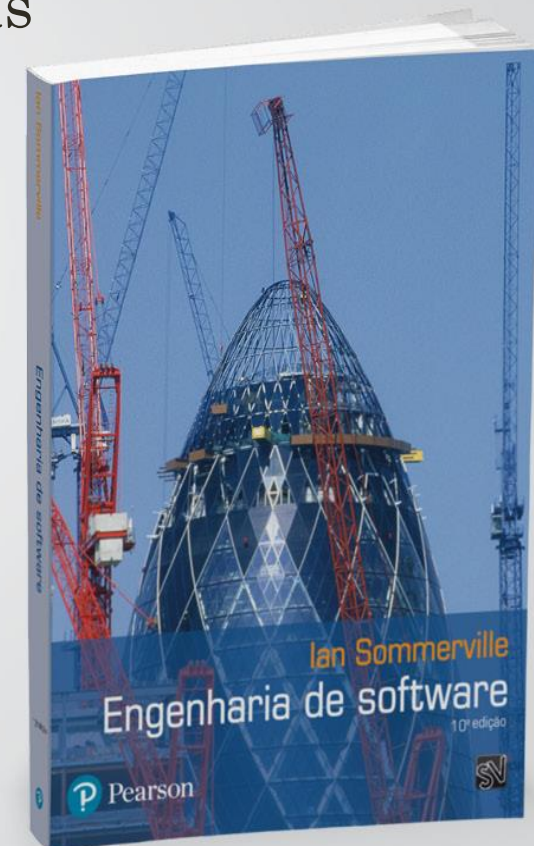


Engenharia de Software

Prof^a. Cristiane Aparecida Lana
Email: cristiane.lana@univicosa.com.br

AGENDA

1. Software e suas características
2. Engenharia de Software e as forças propulsoras
3. Processo de desenvolvimento de software
4. Desafios no desenvolvimento de software
5. Qualidade de software
6. Engenharia de software e análise de sistemas
7. O começo de tudo...
8. O desenvolvimento de software é uma arte?
9. Ética e responsabilidade profissional
10. Exercício



ENGENHARIA DE SOFTWARE E AS FORÇAS PROPULSORAS

FORÇAS PROPULSORAS DA ENGENHARIA DE SOFTWARE

❑ A importância crescente do papel da manutenção e evolução do software

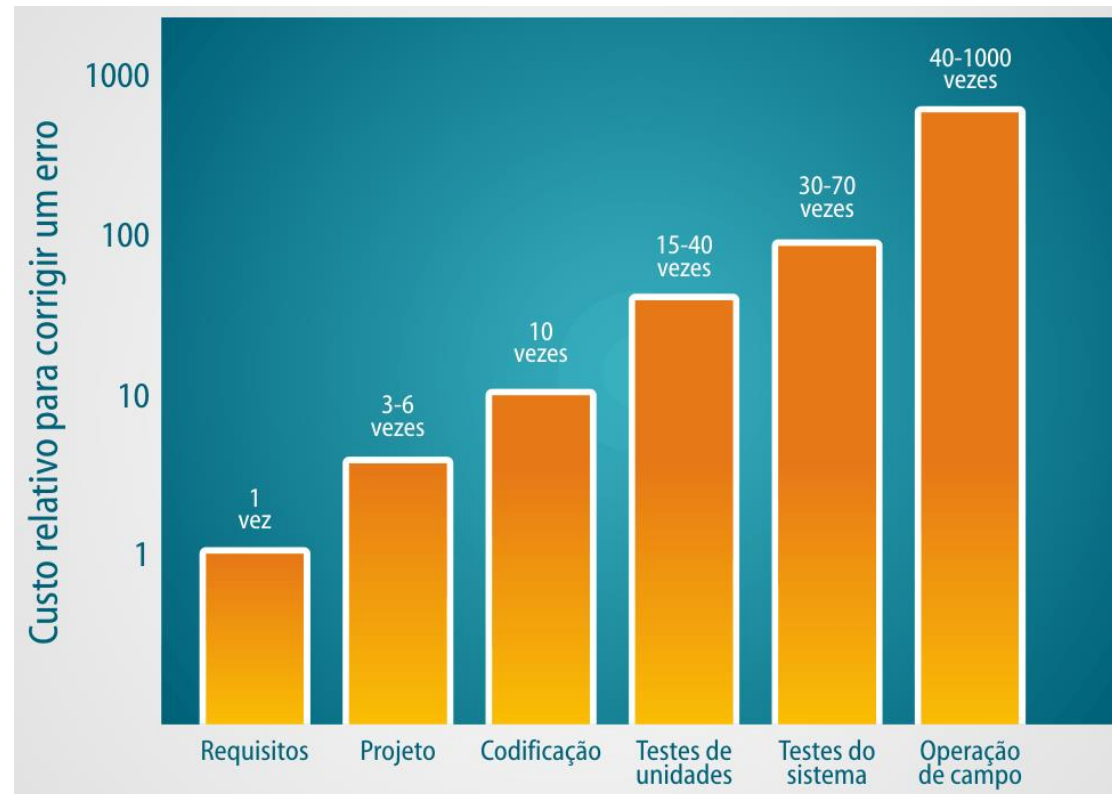
- ✓ correção de erros, modificações, adição de novas funcionalidades.
- ✓ custo de manutenção representa o dobro do custo de desenvolvimento.

❑ Avanços no Hardware e de Técnicas de Programação

- ✓ Tem-se computadores de pequeno porte mais **poderosos** e mais **baratos** do que os antigos **mainframes**.
- ✓ computador passou a fazer parte do dia-a-dia das pessoas, sendo interativos, online, multiusuários, tempo real, etc.

FORÇAS PROPULSORAS DA ENGENHARIA DE SOFTWARE

- ❑ **A importância crescente do papel da manutenção e evolução do software**
 - ✓ correção de erros, modificações, adição de novas funcionalidades.
 - ✓ custo de manutenção representa o dobro do custo de desenvolvimento.



FORÇAS PROPULSORAS DA ENGENHARIA DE SOFTWARE

- **Demanda crescente por programas**
 - ✓ uso de **computador em diversas áreas**, tais como, medicina, educação, entretenimento, editoração, etc
- **Demanda por programas maiores e mais complexos**
 - Efeito bola de neve: **avanços tecnológicos**
 - Avanços nos programas
 - Aumento da demanda por programas
 - Novos avanços tecnológicos.

FORÇAS PROPULSORAS DA ENGENHARIA DE SOFTWARE?

- ✓ Técnicas usadas para sistemas de pequeno porte não funcionavam para sistemas de grande porte e complexos.
- ❖ **Resultado:** estouro de orçamentos, atraso nos prazos, programas não-confiáveis e projetos abandonados.

PROCESSO DE DESENVOLVIMENTO DE SOFTWARE — VISÃO GERAL

PROCESSO DE ENGENHARIA DE SOFTWARE

O QUE É PROCESSO?

- É uma ordenação específica de atividades com clara identificação de entradas e saídas que fornecem **valor de negócio**.

PROCESSO DE ENGENHARIA DE SOFTWARE

ELEMENTOS DE UM PROCESSO



O PROCESSO DE ENGENHARIA DE SOFTWARE

- ❖ Um conjunto estruturado de atividades necessárias para desenvolver um sistema de software.
- ❖ Um modelo de processo de desenvolvimento de software é:
 - ❖ uma representação abstrata de um processo.
 - ❖ Ele apresenta uma descrição do processo de uma perspectiva em particular.

PROCESSO DE ENGENHARIA DE SOFTWARE

ALGUMAS RAZÕES PARA MODELAR PROCESSOS

- Reduzir o tempo de treinamento e o nível de abstração
- Melhora a comunicação e coordenação
- Aumenta produtividade
- Padronização
- Facilita o processo de governança
- Melhora o desempenho do processo
- Reduz risco

PARADIGMAS DA ENGENHARIA DE SOFTWARE

CICLO DE VIDA DO SOFTWARE

- ❖ O ciclo de vida de um software é similar à da vida humana
- ❖ Não existe um limite claro e preciso entre as fases de criança, adolescente, adulto e idoso.
- ❖ E esses limites também não são relevantes na ES.

PARADIGMAS DA ENGENHARIA DE SOFTWARE

CICLO DE VIDA DO SOFTWARE

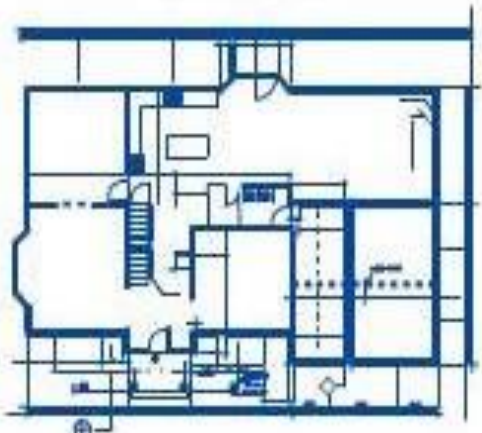
- ❖ Embora exista **vários processos** de desenvolvimento de **software diferentes** todos envolvem:
 - ❖ **Análise de Requisitos e Especificação**
 - ❖ **Projeto**
 - ❖ **Implementação**
 - ❖ **Validação**
 - ❖ **Evolução**

A diferenciação de **uma fase para outra** não são claramente definidas

PARADIGMAS DA ENGENHARIA DE SOFTWARE

Engineering:

Design



Build

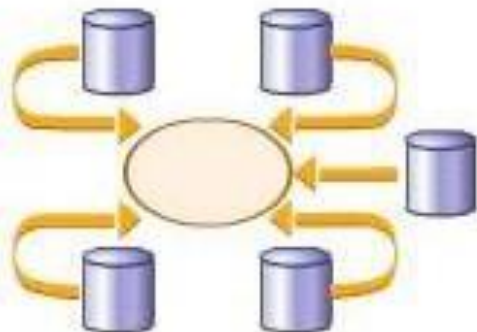


Product



Software Engineering:

Design



Build



Product



PARADIGMAS DA ENGENHARIA DE SOFTWARE

❖ **Análise de Requisitos e Especificação:**

- Análise, **entendimento e registro do problema** que o cliente está tentando resolver.
- Funções, **metas e restrições** do sistema proposto devem ser especificados com precisão.
- Cliente e os desenvolvedores do sistema devem **concordar com a especificação**.

PARADIGMAS DA ENGENHARIA DE SOFTWARE

❖ Projeto:

- consiste no projeto de uma solução que atenda a especificação delineada na fase de análise e especificação

PARADIGMAS DA ENGENHARIA DE SOFTWARE

❖ Implementação:

- consiste basicamente **na geração de código** para os componentes do sistema.
- as **três atividades** da implementação são: **codificação**, **teste de unidade** e **integração**.
 - **codificação:** consiste **em escrever o código fonte** para alguma unidade modular.
 - **teste de unidade:** consiste em **submeter a unidade codificada** a **uma série de casos de teste** para validação da implementação.
 - **integração:** consiste em **agrupar unidades** previamente **testadas** e **testá-las novamente** como um subsistema.

PARADIGMAS DA ENGENHARIA DE SOFTWARE

❖ Teste de Sistema

- consiste em **testar o sistema** como **um todo** (com todas as unidades integradas) para **verificar se ele atende** aos **requisitos** definidos na fase de análise e especificação.

❖ Instalação

- a instalação pode **ser feita pelo próprio cliente** (caso geral para microcomputadores) ou **pelo desenvolvedor**.
- o cliente é **responsável por testar o sistema** e **verificar** se ele é aceitável (**teste de aceitação**).

DESAFIOS NO DESENVOLVIMENTO DE SOFTWARE

DESAFIOS NO DESENVOLVIMENTO DE SOFTWARE

- **Software tornou-se pervasivo** (**incorporado**) no comércio, na cultura e em nossas atividades cotidianas.
- **O processo de desenvolvimento de um software** deve **ser adaptável e ágil**, atendendo **às necessidades** daqueles que usarão o produto.
- Embora a **indústria caminhe** para a **construção** com base **em componentes**, a maioria dos softwares continua **a ser construída** de forma **personalizada** (sob encomenda).

DESAFIOS NO DESENVOLVIMENTO DE SOFTWARE

- ❖ Natureza Sequencial do Desenvolvimento do Sistema
 - mesmo quando paralelismo de fases é possível,
 - adicionar mais pessoas ao projeto pode ser contraproducente
 - pois aumentam os caminhos de comunicação.
 - conceito Homem/Mês (agora pessoa/mês)

DESAFIOS NO DESENVOLVIMENTO DE SOFTWARE

○ Características do Projeto

- nenhum projeto é **tão fácil quanto parece** => otimismo inicial
- **quanto maior e complexo** o projeto => mais difícil
- **sistemas inéditos** (desconhecidos) => mais difícil
- **requisitos** que **mudam** ao longo do projeto
- Diferentes **níveis de segurança**, tempo de resposta, **confiabilidade**, etc.

DESAFIOS NO DESENVOLVIMENTO DE SOFTWARE

❖ Questões de gerência

- troca de gerentes
- incompetência
- equilíbrio dos aspectos técnicos e gerenciais

DESAFIOS NO DESENVOLVIMENTO DE SOFTWARE

❖ Questões de gerência

- troca de gerentes
- incompetência
- equilíbrio dos aspectos técnicos e gerenciais

❖ Entrega

- O desafio da entrega consiste em **diminuir** os **tempos** de **entrega** dos **sistemas grandes e complexos**,
- sem **comprometer** a sua **qualidade**

DESAFIOS NO DESENVOLVIMENTO DE SOFTWARE

Confiança

Software está **entrelaçado** com todos os **aspectos da nossa vida**, é essencial que **possamos confiar** nele.

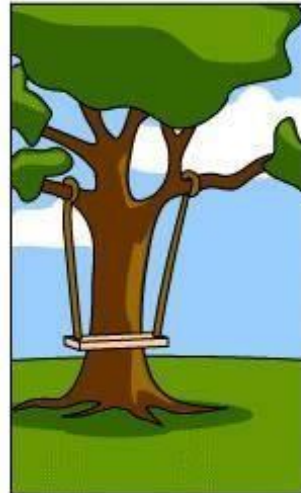
O desafio da confiança é **desenvolver técnicas** que **demonstrem** que o **software** pode ter **a confiança** dos seus usuários.



DESAFIO DA COMUNICAÇÃO



Como o cliente explicou o que queria



Como o Gerente de Projeto entendeu.



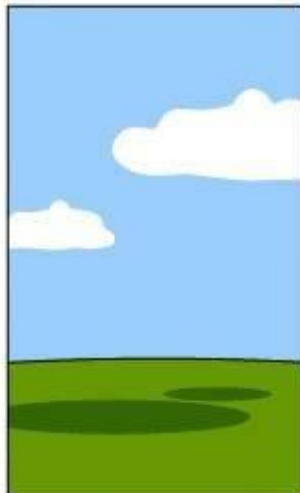
Como o Analista fez o projeto.



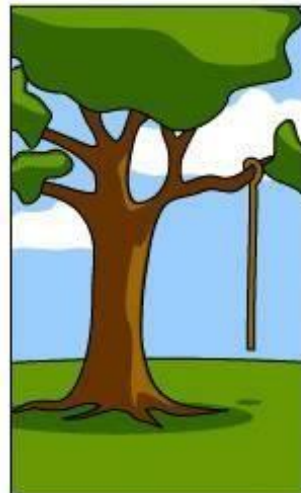
Como o programador codificou (programou).



Como o Vendedor (Consultor de negócios) vendeu.



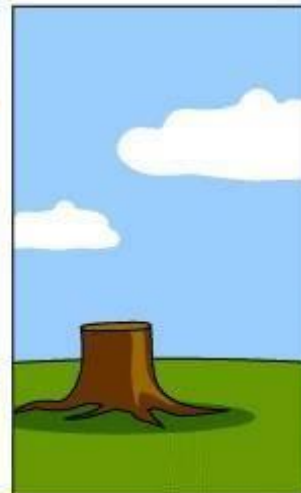
Como o projeto foi documentado.



O que foi colocado em operação.



O que o cliente pagou.



Como é o suporte.



O que o cliente realmente queria.

DESAFIOS DA COMUNICAÇÃO

❖ Comunicação em desenvolvimento dirigidos a planos:

- entre cliente e desenvolvedor
- entre equipes de desenvolvimento
- deve ser feita com documentação => formalismo
- ao final de cada fase um documento escrito e padronizado deve ser produzido => pessoas vão e vêm mas o projeto FICA.

DESAFIOS DA COMUNICAÇÃO

❖ Comunicação em projetos ágeis:

- entre cliente e desenvolvedor
- entre equipes de desenvolvimento (*daily meeting*)
- deve ser feita diariamente face a face (F2F) => sem documentação
- Os *sprints* devem ser minimamente documentados => pessoas vão e vêm mas o projeto FICA.

DESAFIOS DA COMUNICAÇÃO

- Normalmente, os analistas e usuários não sabem no início do desenvolvimento todos os requisitos do sistema, eles são mutáveis ao longo do tempo.
 - Exemplo: mudanças no modelo de negócio da organização que contratou o sistema.
- Assim, o fator determinante para a boa qualidade da análise do sistema
- É a boa interação (comunicação) entre os usuários e os analistas de sistemas.

DESAFIOS DE COMUNICAÇÃO

❖ Como superar problemas de comunicação?

- Usar linguagens de especificação de sistemas, que podem ser entendidas tanto por analistas quanto por usuários.
- As linguagens (entre elas UML) permite discutir *modelos* do sistema com os usuários.
- Usar prototipação para facilitar a compreensão de requisitos complexos

OUTROS DESAFIOS

- Lidar com o aumento da diversidade do software.
- Maior demanda e pressão por um menor tempo de desenvolvimento e entrega do software.
- Desenvolvimento de software confiável independente da complexidade
- Software para web e sistemas distribuídos.
- Software para dispositivos móveis.
- Aumento da preocupação com a questão “segurança” e LGPD

OUTROS DESAFIOS

- ❖ Sistemas Legados
- ❖ Diferentes tecnologias
- ❖ Pressão do tempo
- ❖ Qualidade x Rapidez



QUALIDADE DO SOFTWARE

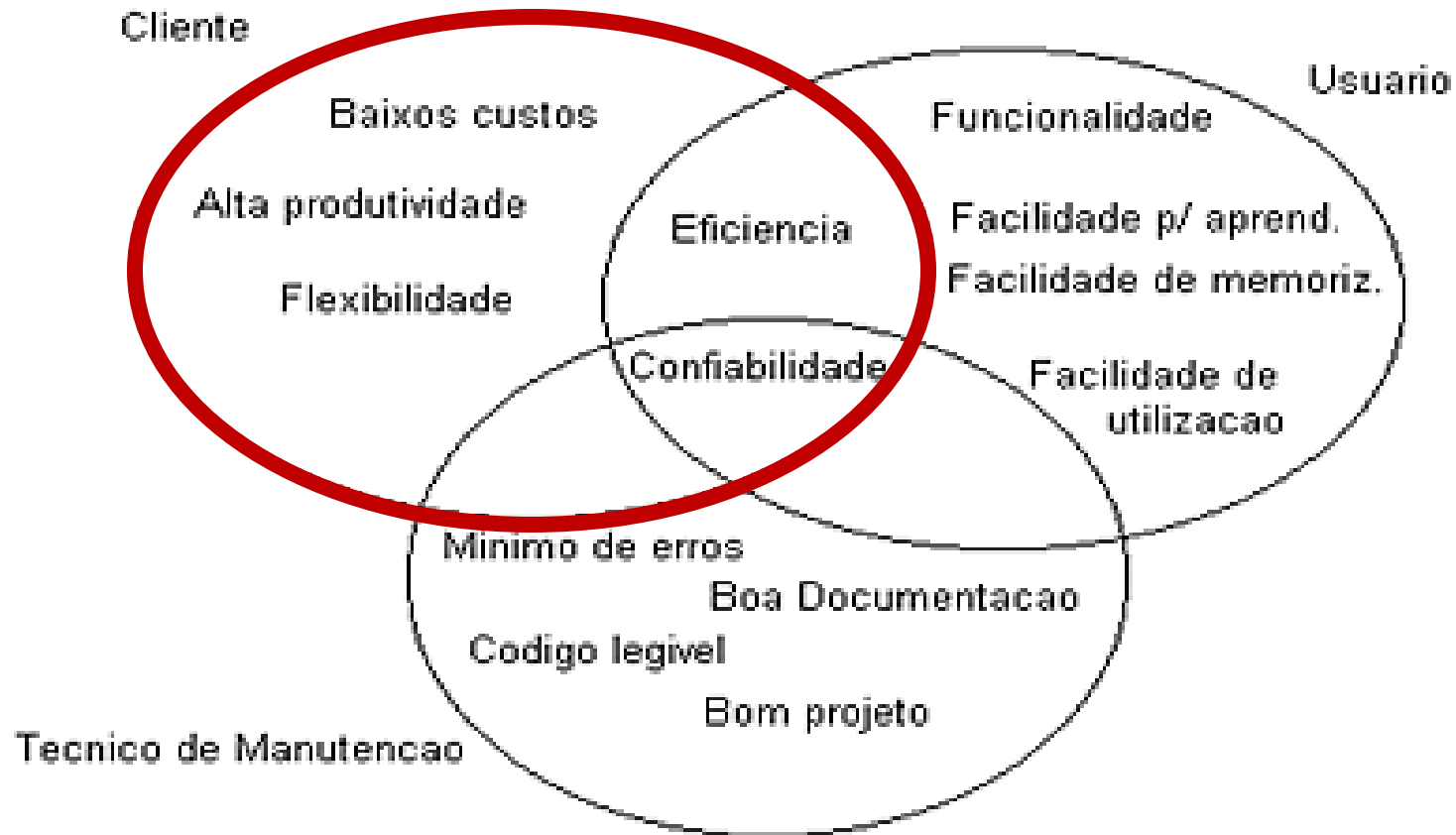
PROCESSO DE ENGENHARIA DE SOFTWARE

SISTEMAS COM QUALIDADE É UMA META PRIMÁRIA DA ENGENHARIA DE SOFTWARE

- ❖ O que significa um sistema com qualidade?
 - depende de quem vai dar a resposta
- ❖ Existem 3 categorias de pessoas para as quais a qualidade do sistema é importante:
 - **Cliente (sponsor):** pessoa ou organização que esta pagando pelo desenvolvimento do sistema e que será responsável por qualquer custo futuro.
 - **Usuário:** qualquer pessoa que venha a usar o sistema.
 - **Técnico:** pessoas responsáveis pelas correções e modificações após instalação.

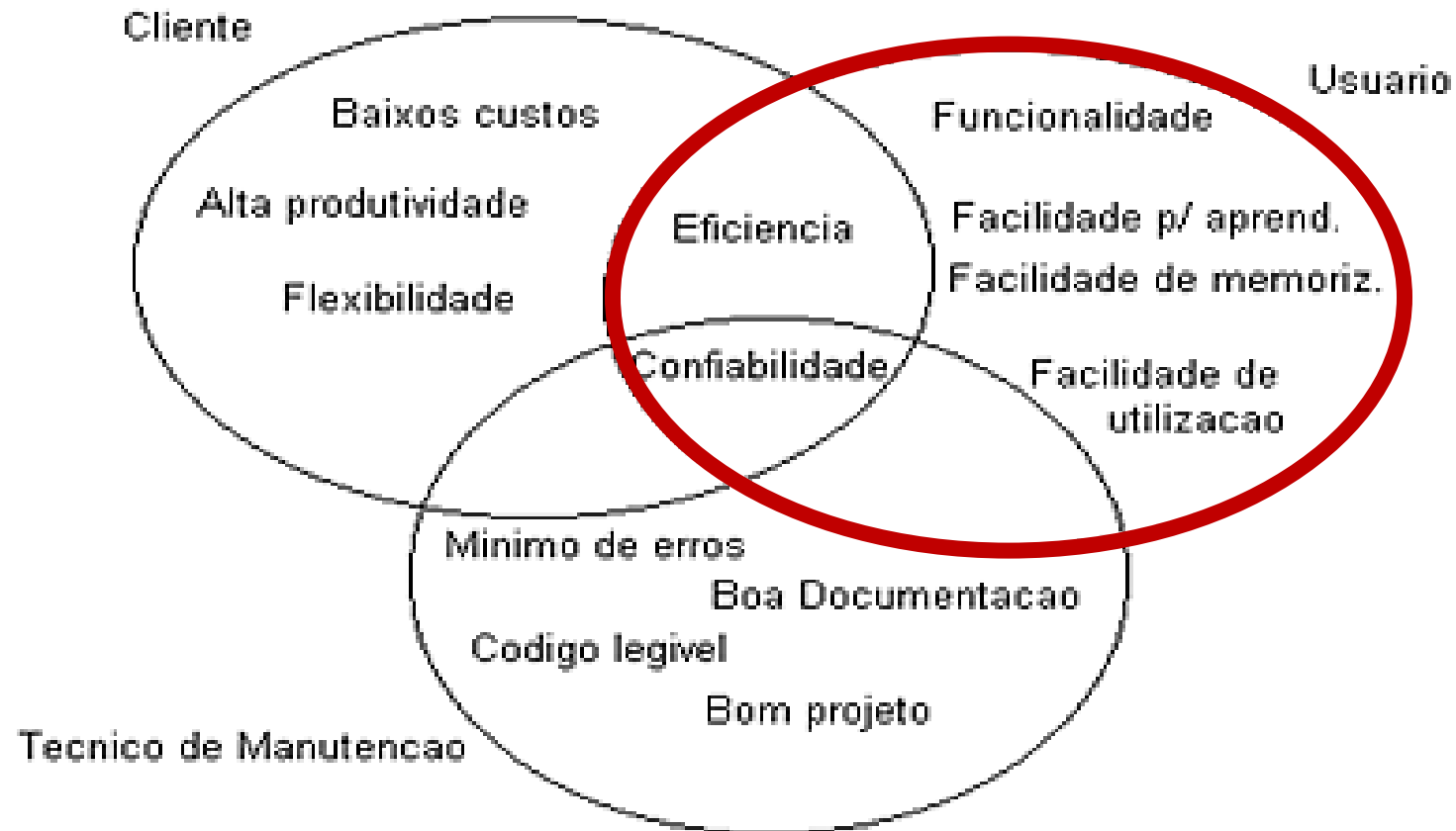
QUALIDADE DE SOFTWARE

QUALIDADE DO SISTEMA SEGUNDO TRÊS PONTOS DE VISTAS



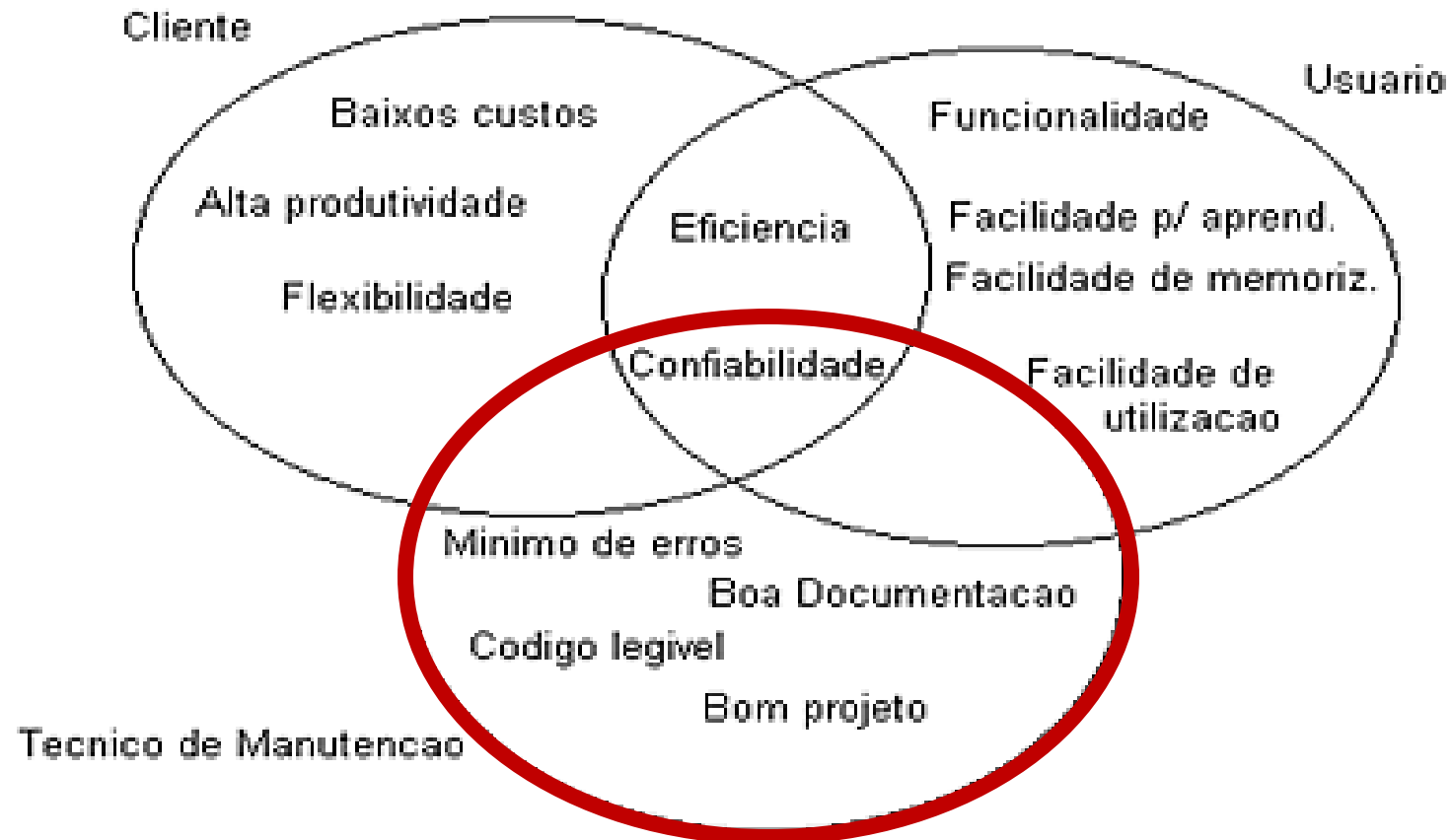
QUALIDADE DE SOFTWARE

QUALIDADE DO SISTEMA SEGUNDO TRÊS PONTOS DE VISTAS



QUALIDADE DE SOFTWARE

QUALIDADE DO SISTEMA SEGUNDO TRÊS PONTOS DE VISTAS

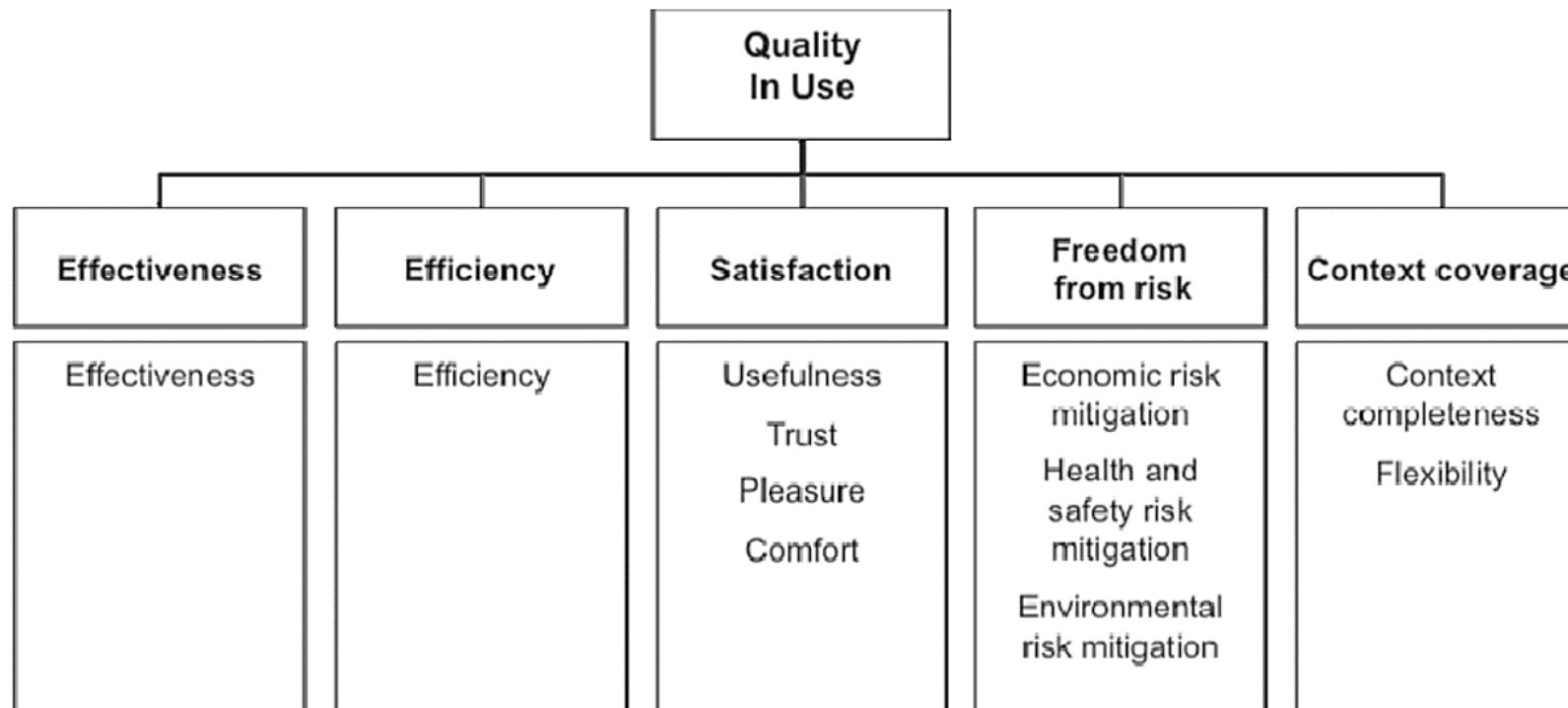


CRITÉRIOS DE QUALIDADE DE SOFTWARE

ISO/IEC 25010:2011

- Um conjunto de atributos relacionados com o esforço necessário para o uso de um sistema interativo, e relacionados com a avaliação individual de tal uso, por um conjunto específico de usuários

Sendo
Revisada



CRITÉRIOS DE QUALIDADE DE SOFTWARE

ISO/IEC 25019:2023

**Quality in use
2023**

Effectiveness

Effectiveness

Efficiency

Efficiency

Satisfaction

Userfulness
Trust
Pleasure
Confort

**Freedom from
risk**

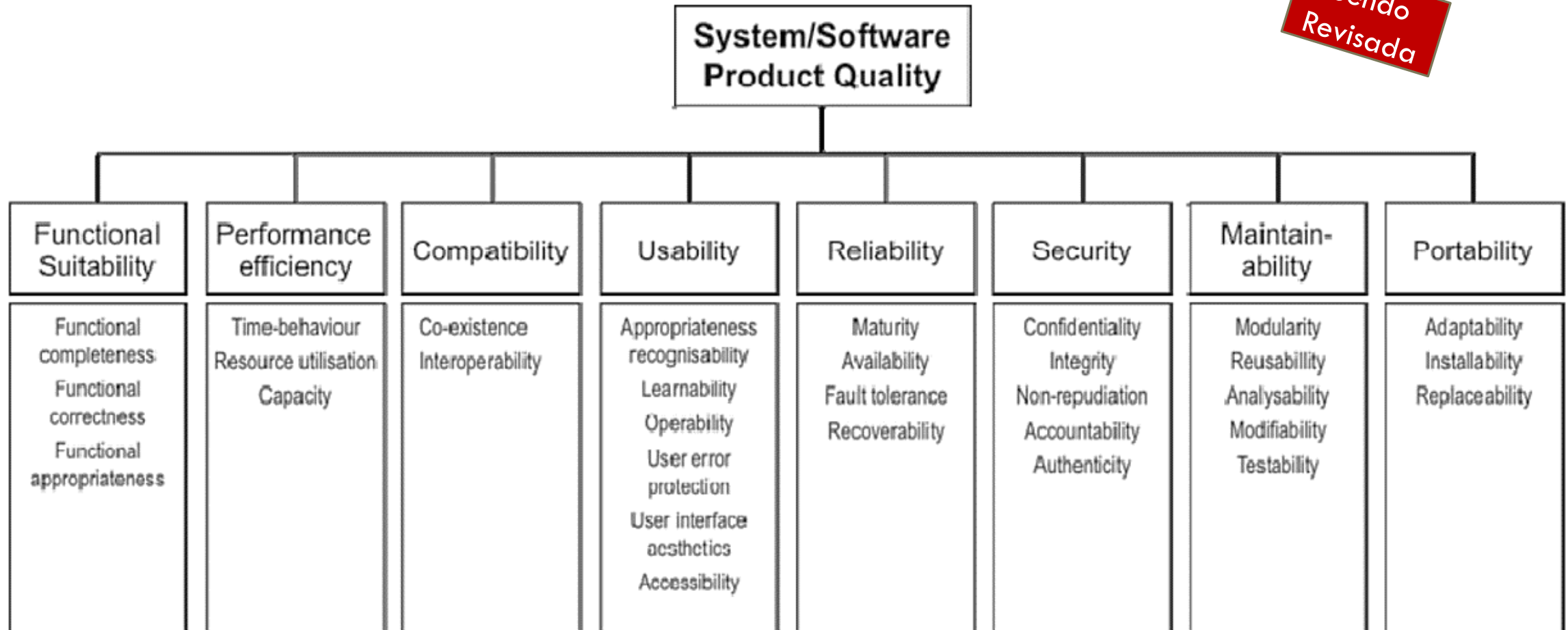
Economic risk
mitigation
Health and safety risk
mitigation
Environmental risk
mitigation

Revisada

CRITÉRIOS DE QUALIDADE DE SOFTWARE

ISO/IEC 25010:2011

Sendo
Revisada



CRITÉRIOS DE QUALIDADE DE SOFTWARE

ISO/IEC 25010:2023

Revisada

Systems/Software Product Quality - 2023

Functional Suitability

Functional completeness
Functional correctness
Functional appropriateness

Performance Efficiency

Time behaviour
Resource utilization
Capacity

Compatibility

Co-existence
Interoperability

Interaction Capability

Appropriateness
recognizability
Learnability
Operability
User error protection
Inclusivity
User assistance
User engagement
Self-descriptiveness

Reliability

Faultlessness
Availability
Fault tolerance
Fault tolerance

Security

Confidentiality
Integrity
Non-repudiation
Accountability
Authenticity
Resistance

Safety

Operational Constraint
Risk identification
Fail safe
Hazard warning
Safe integration

Maintainability

Modularity
Reusability
Analysability
Modifiability
Testability

Flexibility

Adaptability
Installability
Replaceability
Scalability

QUALIDADE DE SOFTWARE

❖ Propriedades emergentes:

- ❖ são **propriedades do sistema** como **um todo** que somente **emergem** quando **todos os sub-sistemas** estiverem integrados.

❖ Exemplos de propriedades emergentes:

- Confiabilidade
- Manutenibilidade
- Performance
- Usabilidade
- Segurança

ALGUNS ATRIBUTOS ESSENCIAIS DE UM SOFTWARE

- **Manutenibilidade** (aborda a **manutenção** do software)
- **Evolucionabilidade** (**capacidade de evolução** para atender novas demandas)
- **Confiabilidade** (Probabilidade de **ocorrência de falhas**)
- **Proteção e Segurança** (integridade, não-repúdio*, autenticidade, etc)
- **Eficiência** (tempo de resposta, tempo de processamento, uso de recursos (memória, rede, armazenamento, etc.))
- **Compreensibilidade e Usabilidade**

* Autoria de uma ação não pode ser identificada

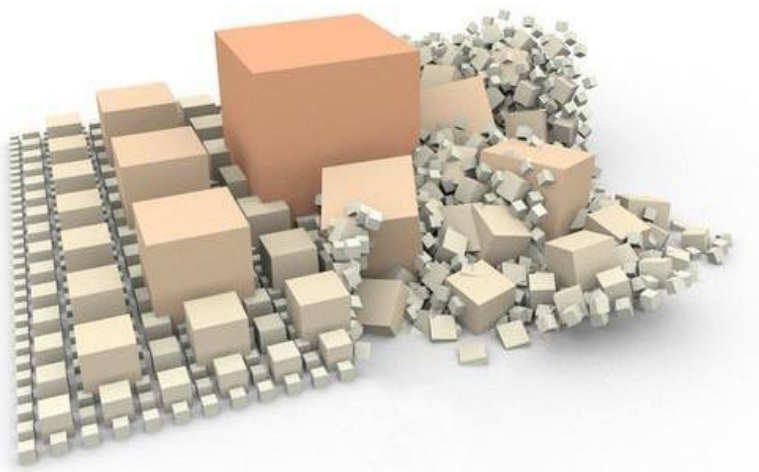


ALGUNS PONTOS IMPORTANTES DO USO DA ES

- ❖ Devemos ser capazes de
 - ❖ produzir sistemas confiáveis e rápidos.
- ❖ Geralmente,
 - ❖ é mais barato, no longo prazo, usar métodos e técnicas de engenharia de software para os sistemas de software em vez de apenas escrever os programas como se fosse um projeto de programação pessoal.
- ❖ Para a maioria dos tipos de sistemas,
 - ❖ a maior parte dos custos são os custos de alterar o software em uso.

A ENGENHARIA DE SOFTWARE

Traz a **ordem** para uma atividade que pode facilmente se **tornar caótica**!



Diminuir o retrabalho

- minimizando o **número de falhas** no projeto que **não foram descobertas** antes da liberação do software
 - **Em resumo:** Traz **resultados** mais **positivos** tanto para **quem desenvolve** o software quanto para **quem o adquirem**.

FIQUE ATENTO!!!

Sempre que você pensar que **não tem tempo** para a atividade de engenharia de software, pergunte a si mesmo: **Teremos tempo para fazer tudo de novo?**



Roger Pressman

ENGENHARIA DE SOFTWARE E ANÁLISE DE SISTEMAS

ENGENHARIA DE SOFTWARE E ANÁLISE DE SISTEMAS

- Quem são os donos da informação?
 - Dados, quando representam **informações importantes**, são fonte de **disputa de poder** em uma empresa.
- Questões:
 - Como identificar os níveis da organização (Estratégico, Gerencial e Operacional) **para a qual se desenvolve o sistema?**
 - Qual seria a **solução** para se **ter acesso** as **informações relevantes** e **gerenciá-las** para o desenvolvimento do software?

ENGENHARIA DE SOFTWARE E ANÁLISE DE SISTEMAS

- Solução:

- Produzir especificações (documentos) de qualidade, dos sistemas de informação que serão criados.
- Seguir metodologias de desenvolvimento de sistemas, que proporcionem:
 - **Produtividade** da equipe de desenvolvimento
 - **Confiabilidade** no sistema
 - **Manutenibilidade** e **portabilidade** do código-fonte
 - **Segurança** dos dados
 - Bom **desempenho**

ENGENHARIA DE SOFTWARE E ANÁLISE DE SISTEMAS

- Solução:

- Ter um profissional que, além do conhecimento técnico
- tenha um perfil também voltado para a especificação dos requisitos do sistema e
- percepção do negócio da empresa.

ENGENHARIA DE SOFTWARE E ANÁLISE DE SISTEMAS

- **Quem** é o profissional que realiza esse trabalho?
 - O **Analista de Sistemas!**

ANALISTA DE SISTEMAS

❖ Habilidades necessárias a um Analista:

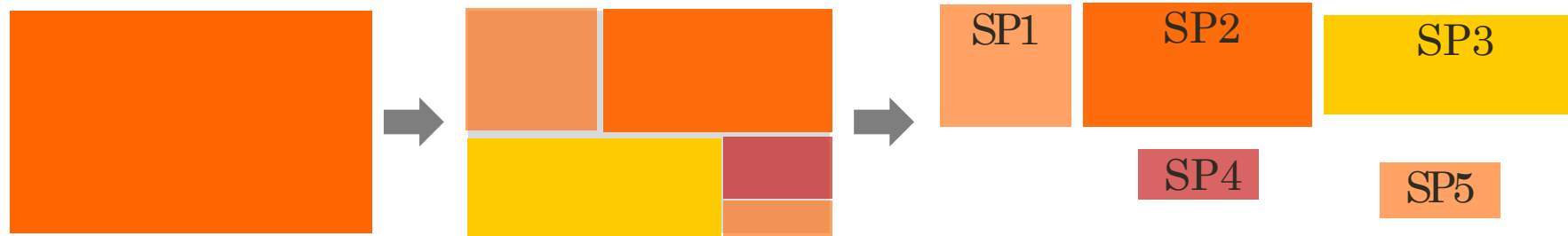
1. **Comunicação:** capacidade de ouvir, redigir e expor ideias com facilidade, clareza e precisão.
2. **Capacidade de Análise:** aptidão para realizar operações mentais com abstrações sobre a realidade em estudo e de estabelecer uma visão sistêmica e crítica do contexto em análise.
3. **Conhecimento da área usuária:** habilidade de absorver conhecimento por meio da vivência com as operações da empresa (que pode ser obtido por meio de entrevistas), ou por meio de leitura de informações técnicas, documentos da empresa, etc.

ANALISTA DE SISTEMAS

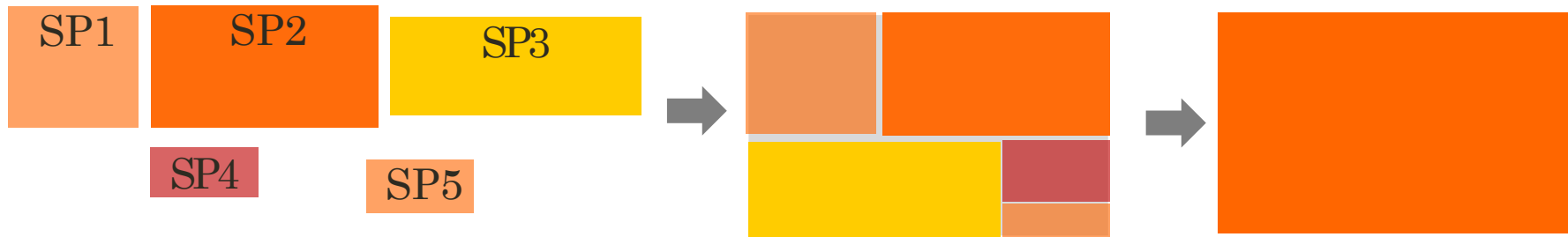
4. **Capacidade de negociação:** habilidade de obter o resultado desejado, administrando situações de conflitos de interesses pessoais, fazendo convergir interesses opostos.
5. **Administração de Projetos:** habilidade de saber alocar tempo e recursos humanos, financeiros e materiais necessários aos projetos.
6. **Conhecimento técnico:** capacidade de especificar sistemas, principalmente em suas fases de nível de abstração mais alto, usando alguma linguagem ou ferramenta de modelagem.

ANÁLISE DE SISTEMAS

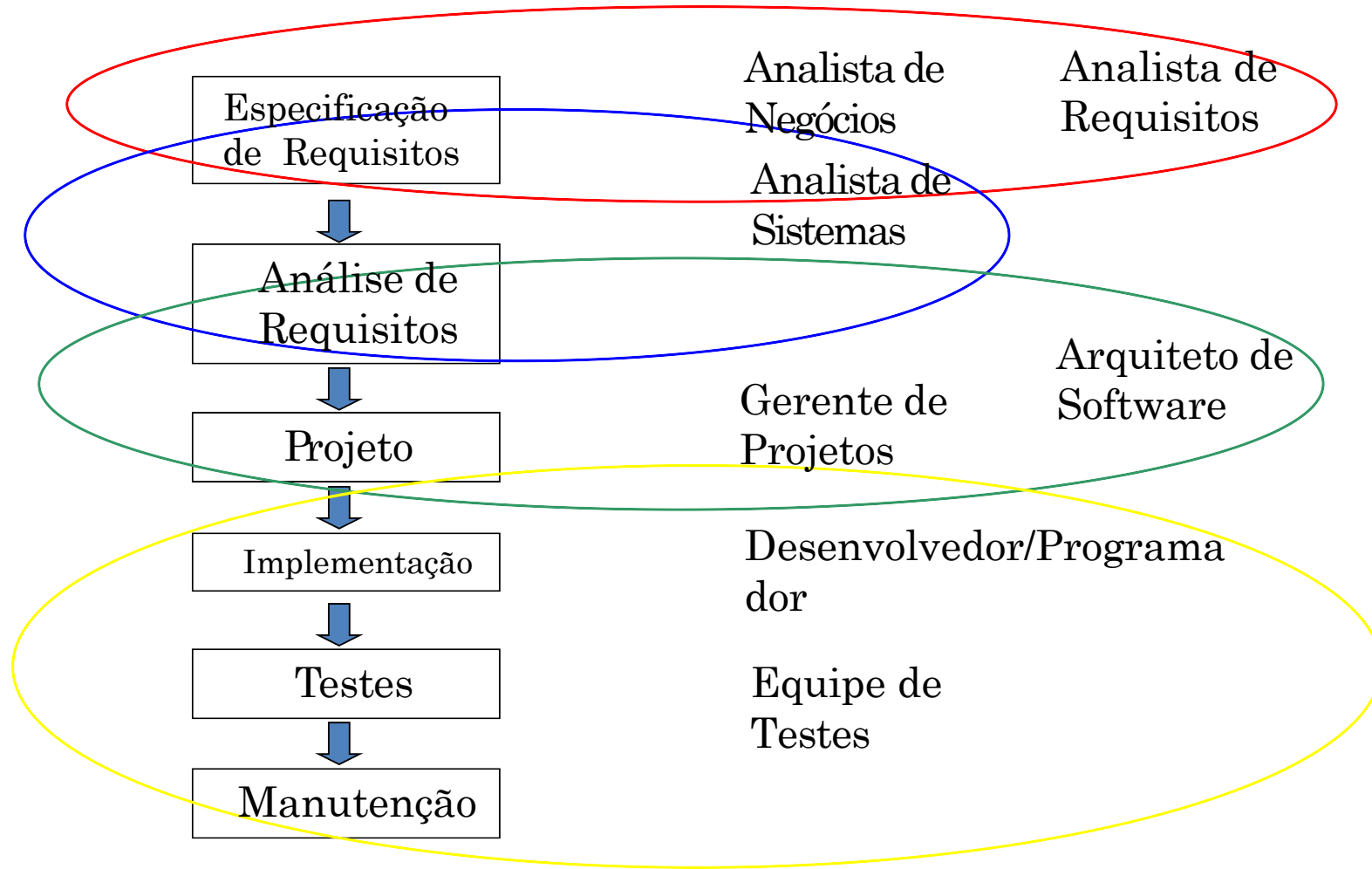
▪ Análise



▪ Síntese



ENGENHARIA DE SOFTWARE - PAPEIS



O COMEÇO DE TUDO...

COMO TUDO COMEÇA?

Todo projeto de software é iniciado por **alguma necessidade** do negócio:

- **Corrigir um defeito** em uma aplicação existente;
- **Adaptar um sistema herdado** (legado) para mudar o ambiente do negócio;
- Necessidade de **estender as funções e características** de uma aplicação existente;
- Necessidade de **um novo produto, serviço ou sistema**.



TIPO DE SOFTWARE

❖ Software de Sistema:

- Coleção de programas escritos para servir outros programas.
- Características:
 - Interação intensa com hardware do computador
 - Uso intenso por múltiplos usuários
 - Compartilhamento de Recursos
- Exemplos:
 - Compiladores, editores e componentes de sistemas operacionais

TIPO DE SOFTWARE

❖ Software Comercial:

- Processamento de [informações comercial](#)
- Para facilitar [operações comerciais](#) ou [tomada de decisão](#) de gestão de negócios
- **Exemplo:**
 - Folha de pagamentos,
 - contas a pagar/receber ,
 - controle estoque e outros.

TIPO DE SOFTWARE

❖ Software Científico e de engenharia:

- Esse tipo de software é caracterizado por processar números
- Simulação de sistemas
- Exemplo:
 - Astronomia
 - biologia molecular

TIPO DE SOFTWARE

❖ Software para Inteligência Artificial

- Faz uso de algoritmos não numéricos para resolver problemas complexos que não são passíveis de computação ou análise direta.
- **Exemplo:**
 - Sistemas Especialistas, também chamados sistemas baseados em conhecimento,
 - Sistemas de reconhecimento de padrões (de imagem e de voz)
 - Sistemas inteligentes em geral

TIPO DE SOFTWARE

❖ Outros

- ❖ E-commerce (B2B, B2C, B2M, e outros*)
- ❖ Aplicações móveis (iPhone, iPad, Tablets em geral, Android)
- ❖ Jogos e entretenimento
- ❖ Robótica
- ❖ Redes sociais
- ❖ etc

* B2B – Negócios para Negócios; B2C – Negócios para Cliente; B2M – Negócios para Muitos (aqui existe outras variações como Business to Me, Mobile Business, entre outros.

SOFTWARE – MUNDO REAL

- ❖ Fatores que devem ser considerados no desenvolvimento/evolução de software do “mundo real” :
 1. Escopo
 2. Custo
 3. Prazo
 4. Qualidade
- ❖ Dependendo do tamanho do software, tais fatores se tornam mais difíceis de garantir.

**O DESENVOLVIMENTO DE
SOFTWARE É UMA ARTE?**

SOFTWARE = ARTE?

❖ Parte arte, parte engenharia...

- Se o cantor/ator/pintor errar, a audiência fica chateada
- Se o engenheiro civil errar o prédio pode cair
- Se o médico errar o paciente pode morrer

❖ Caso o desenvolvedor/equipe de software errar(em), o que pode acontecer?

TRAGÉDIAS ENVOLVENDO SOFTWARE

1º Caso real

Ariane 5 - Foguete lançador de satélites

Problema

- O foguete se autodestruíu 40 segundos após o lançamento

Principais causas

- Software reutilizado sem ser adaptado para o novo hardware.
- Ausência de testes em solo deste software;
- Defeito apresentado em voo.

Consequências

- Prejuízo de mais de US\$ 370.000.000,00 em 1996



TRAGÉDIAS ENVOLVENDO SOFTWARE

2º Caso real

Therac-25

- Therac-25 era uma máquina de radioterapia controlada por computador

Problema

- Emissão de doses indevidas de radiação;

Principais causas

- O código do software não havia sido revisado/testado;
- O projeto do software não havia sido documentado em detalhes suficientes para permitir o entendimento dos erros;
- A documentação do sistema fornecida aos usuários não explicava o significado dos códigos de erro que a máquina retornava...



TRAGÉDIAS ENVOLVENDO SOFTWARE

2º Caso real

Therac-25

Consequências

- Houve uma série de **pelo menos 6 acidentes** entre 1985 e 1987, nos quais os pacientes receberam *overdose* de radiação;
- Pelo **menos cinco mortes** aconteceram **devido aos acidentes**;



TRAGÉDIAS ENVOLVENDO SOFTWARE

3º Caso real

Desastre com Airbus 320

Problema

- Navio *US Vincennes* (atirador de misséis) derrubou um Airbus 320 em 1988
- Guerra Irá-Iraque

Principais causas

- Falha no software de reconhecimento, confundindo o avião com um F-14 Iraniano

Consequências

- 290 mortes



TRAGÉDIAS ENVOLVENDO SOFTWARE

4º Caso real

Game - O Rei Leão animado

Problema

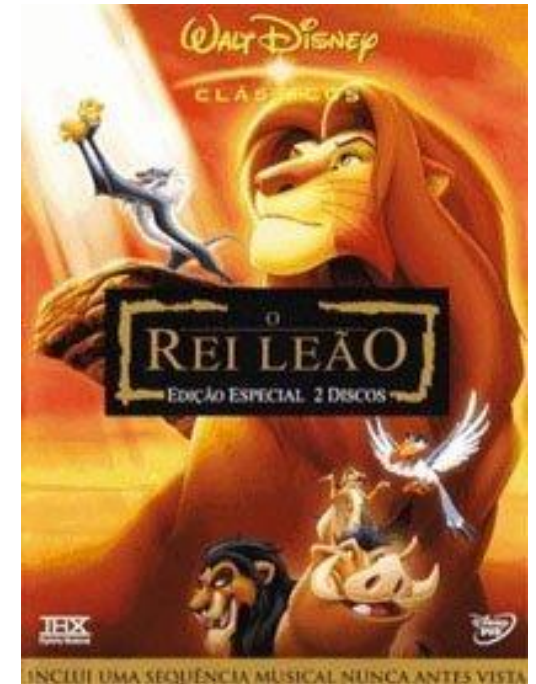
- O jogo **não funcionava** em **uma variedade** de modelos de **computadores**

Principais causas

- Os **desenvolvedores** da *Disney* **testaram o jogo** apenas **naqueles computadores** próximos aos **que os eles usaram** no desenvolvimento.

Consequências

- Enorme **prejuízo para empresa**, com **devolução de dinheiro** aos clientes e **reformulação** do game.



REFERÊNCIAS

1. Sommerville, I.. Engenharia de Software. 10ed, Ed. Pearson, 2019. ISBN: 9788543024974.
2. Pressman, R. S.; Maxin, B. R..Engenharia de Software: uma abordagem profissional. 9ed, AMG Editora Ltda: Porto Alegre. 2021.
3. Wazlawick, R.S., Análise e Projeto de Sistemas de Informação Orientados a Objetos, GEN LTC, 3ª edição, 2014.
4. Guedes, G. T. A. UML 2 - Uma Abordagem Prática – 3ed. Novatec, 2018. ISBN: 9788575226469



Engenharia de Software

Prof^a. Cristiane Aparecida Lana
Email: cristiane.lana@univicosa.com.br